Université Saint-Joseph de Beyrouth
Faculté d'ingénierie

Final Year Project description form

Project duration: 4 months (from 19th of January till May 12th 2026)

1. **Reference code (By Admin)**:

FYP_26_21

2. **Supervisor***:

Name: Anthony Assi Phone number: +1-415-521-0389 Email: anthony.assi@gmail.com Professional location: San Francisco, California

3. **Co-supervisors (or industrial supervisor)**

Name: Phone number: Email: Professional location:

4. **Student names** (if applicable)

5. **Sponsor**

6. **Project title***

AI Coding Agent: An Autonomous LLM Agent for Proactive Code Generation and Reactive Debugging

7. **Project objectives*:** (1-2 sentences to explain the objectives of the project)

This project's objective is to build a dual function autonomous agent: first, to proactively generate runnable Python scripts from high level natural language requirements (e.g., "write a software in python that gets the displays on the screen the latest Microsoft stock price"). Second, to reactively debug existing broken code or local system dependencies by iteratively executing it, analyzing errors, and applying automated fixes until it runs successfully on the local environment with no human interventions.

8. **Project description***: context, contemporary issues, impact…(The project should have a major design experience based on the knowledge and skills acquired in earlier course work)

The developer workflow is currently being revolutionized by GenAI and LLMs. However, this workflow is bifurcated: developers use one process for generating new code (prompting on GenAI platforms such as ChatGPT) and a separate, manual process for debugging it when it fails on their targeted host. This project consists of developing a truly autonomous agent that must be able to handle the full lifecycle: from idea to functional code. This includes planning, writing, and most importantly running and fixing its own-generated code on the local machine. Furthermore, this same debugging logic can be isolated and applied to any existing broken script, automating the tedious "copy-paste-error-fix" loop. This project creates a single, powerful tool that serves as both a "code generator" and a "code fixer."

The project will consist of the below 8 Phases:

Phase 1. Agent Technology Analysis and Project Scoping

- Conduct a comprehensive market analysis of existing tools, comparing their capabilities and highlighting their respective strengths and weaknesses.
- Define and Scope the Engineering workflow for the two key tasks:
  - **Code Generation (Proactive):** This involves researching agent architectures that prioritize planning and tool use. The analysis will focus on how agents interpret natural language, develop multi-step plans, select appropriate tools (e.g., writing to a file, searching for library documentation), and generate code to fulfill a given request.
  - **Code Debugging (Reactive):** This task requires researching architectures centered on error-driven loops. The focus will be on analyzing how agents parse stderr outputs, classify error types, and map them to corrective actions or "tools" (e.g., running pip install, programmatically editing a line of code).
- The outcome of this phase will be a formal scope document detailing the agent's capabilities and limitations for both code generation and debugging.

Phase 2. Unified System Architecture Definition

- Design a high-level architecture that supports two distinct operational modes: "Generate Mode" and "Debug Mode."
- Identify shared components (LLM Analysis, Filesystems I/O, Sandboxed Shell Tool) and specialized components (e.g., "Generation Planner," "Error Classifier").
- Define the microservice architecture and all foundational components.

Phase 3. Feature 1: Proactive Code Generation

- Implement the "Generate Mode" orchestrator.
- This agent will take a natural language prompt (e.g., "write me a code to display the latest Microsoft stock price").

- It must create a plan, identify necessary libraries (e.g., yfinance), write the code to a file, and attempt to run it.

Phase 4. Agent Self-Correction Loop

- This phase directly follows Phase 3. The agent's generated code will inevitably fail on the first run (e.g., missing library, simple syntax error).
- Develop the "self-correction" loop: the agent must read the error from its own script, analyze it, and apply a fix (e.g., pip install yfinance or edit the file).
- This loop is the critical bridge between the two features.

Phase 5. Feature 2: Standalone Reactive Debugger

- Formalize and isolate the "self-correction" loop from Phase 4 into the standalone "Debug Mode."
- This mode will be invoked with a specific target file (e.g., python ESIBaiAgent.py --fix mybrokencode.py).
- The agent will exclusively run the "Execute-Analyze-Fix" loop, focusing on ModuleNotFoundError, SyntaxError, etc., until the script runs without stderr.

Phase 6. Integration and Evaluation

- Integrate both "Generate Mode" and "Debug Mode" into a single command-line tool.
- Develop a test suite of 10-15 "tasks" (e.g., 5 generation prompts) and 10-15 "broken files" (e.g., 5 syntax errors, 5 import errors).
- Evaluate the agent's success rate, time-to-completion, and types of errors it can or cannot handle.

Phase 7. Code Packaging and Documentation

- Write detailed documentation for the overall solution, explaining the architecture for both modes.
- Provide clean source code that is properly packaged and reusable.
- Provide a How-To documentation that describes how to run the agent in both "Generate Mode" and "Debug Mode."

Phase 8. Project Deliverables:

- **Analysis Report:** A report on the different agent design patterns for generation vs. debugging and the performance of each.
- **Working Code:** The complete, packaged, and documented source code for the agent.
- **Final Report:** A comprehensive report detailing the unified system design, development process, evaluation results for both features, and recommendations for future work.
- **Progress Report:** A mid-term progress report document with a PowerPoint presentation (likely after Phase 3 or 4).
- **Final Presentation:** A final project report document with a PowerPoint presentation and a pre-recorded demo showing *both* features in action.

- **Key Questions Analysis:** A dedicated section in the final report that provides researched answers to the following questions:
  - Question 1: The Future of Junior Developers: When would AI agents take on the job of freshly graduated software engineers? What are the weaknesses of AI agents today? How can junior software engineers stay relevant?
  - Question 2: Economic Viability Analysis: How much should an AI coding agent cost in comparison to a software engineer's salary?

9. **Preliminary functional requirements and constraints***: project requirements are functionalities to be implemented and/or goals that must be reached to ensure the success or completion of the project. Technical and non-technical constraints are restrictions that define the project's limitations[1].

Students should adhere to the requirements and constraints detailed in the project description section above.

1. **Standards Used**: Specify the standards and/or codes relevant to the project. These should be established by the supervisor and further detailed by students through a compliance table outlining specific requirements for each applicable engineering standard. Students are expected to provide comprehensive information, including the standard referenced, the relevant parts or sections implemented, and a well-reasoned justification demonstrating how their solution satisfies each requirement. The following compliance table provides illustrative examples. This table is provided as a guide to help students understand how each standard can be addressed in their project and ensure that the project conforms to all necessary protocols and industry standards.

- Languages: Python
- OS: Ubuntu Linux
- APIs: Hugging Face Transformers for LLM integration, OpenAI API (optional)
- Environment: Google Colab, VSCode, Github.
- Coding standards: GoF Design Patterns, Object Oriented Design, RegEx, JSON format and OpenSource Standards.
- Methodology: Agile

10. **Required tools and critical resources***: list of essential resources needed for the project (software, hardware, equipment, data, etc.)

Resource	**Provided by:** *advisor, partner, sponsor…*	**Estimated cost**	**Notes**

[1] ABET EAC Criteria: For illustrative purposes only, examples of possible constraints include accessibility, aesthetics, codes, constructability, cost, ergonomics, extensibility, functionality, interoperability, legal considerations, maintainability, manufacturability, marketability, policy, regulations, schedule, standards, sustainability, or usability…

Cloud VM	provided by advisor	Free	Logins to be sent to students
LLM API Key	provided by advisor	Free	API Key to be sent to students
Github Repo	provided by advisor	Free	Students to be added to a private github repo

11. **Project deliverables***

Meeting minutes, proposal report, progress report, final report, presentations via Moodle
Add other outcomes when applicable (Drawings, calculus, prototype…)

Refer to the Phase 8. Project Deliverables in the Project description section above.

12. **Number of students*** (specify number of students per option/program: 1 to 4)

Computer and communications engineering, Software engineering option: 4

13. **Prerequisites**: List all required courses or skills for your project

Required Courses:
Machine Learning
Probabilités et statistiques
Bases de données relationnelles
Informatique 3
Programmation Orientée Objets
Administration Unix
Programmation pour le Web

Required skills:
Python, Linux.

14. **Required attendance in Lab or industry (Specify number of days and hours of presence per week onsite or remotely)**

Fully Remote

15. **Date**

25 Oct 2025

Submitted by: *Anthony Assi*

Checked by (all fields answered) FYP coordinator

Approved by Program committee

Approved by Head of Department (Name and Signature)

* *All entries must be filled in this project description.*

** *For more information, please contact ESIB FYP coordinators:*

- *Dr. Rima Kilany Chamoun (01421332, rima.kilany@usj.edu.lb) for FYP related to the CCE program.*
- *Dr. Chantal Maatouk (01421344, chantal.maatouk@usj.edu.lb) for FYP related to the EE and ME programs.*

*** *Form to be returned to the respective FYP coordinators before November 7th, 2025.*